SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 2 – CO2

Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Student Name	**P .Lakshmi Gayathri Navaneetha**	**Register Number**	**192321169**
Department / Section	**IT**	**Academic Year / Semester**	**4 year**
Faculty Name	**Dr. kesavan**	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	**06-11-2026**	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Select Test Case Design Strategies	5		
2	Evaluate Random Testing	2		
3	Design and Evaluate Equivalence Classes	6		
4	Design and Evaluate Boundary Value Tests	4		

5	Apply Other Black-Box Techniques	3		
6	Design and Evaluate State-Transition Tests	3		
7	Construct the Control Flow Graph	4		

8	Evaluate Path Testing and Cyclomatic Complexity	4		
9	Judge Test Adequacy and Coverage	3		
10	Compare Black-Box and White-Box Effectiveness	4		
11	Recommend and Defend the Final Test Suite	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of the Original Suite	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A justified selection of test design strategies for each part, with strengths and weaknesses		
2	A reasoned evaluation of the usefulness and limits of random testing		
3	A complete class table, derived tests with expected results, and the defects they expose		
4	A justified boundary value design with expected results and a defended BVA form		
5	A decision table with derived tests and a comparison with ECP/BVA coverage		
6	A state-transition model with justified valid and invalid transition tests		
7	A correct control flow graph with node–line mapping and code-versus-specification mismatches		
8	A derived cyclomatic complexity, basis paths with test data and loop-test evaluation		

9	A justified adequacy criterion supported by coverage analysis of the student's own tests		
10	A reasoned black-box versus white-box comparison supported by the student's own results		
11	A technically justified final suite supported by black-box, white-box and coverage evidence		
12A–D	Impact analysis, evaluation of the original suite, adaptation or redesign, and an evidence-based defence		

TEST DESIGN EVIDENCE CHAIN (Tasks 3–11)

Evidence item	Status	Observation
Equivalence classes and derived tests with expected results		
Boundary tests and comparison of the forms of BVA		
Decision table and comparison with ECP/BVA		
State-transition model and risk-based invalid tests		
Control flow graph with node–line mapping		
Cyclomatic complexity (two methods) and basis paths with test data		
Coverage achieved and tests for the next stronger criterion		
Black-box versus white-box comparison from own results		
Final suite traceable to techniques, with run time against the limit		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Expected results calculated from the specification, not from the code	☒ Yes ☐ Partly ☐ No
Cyclomatic complexity shown by more than one method	☒ Yes ☐ Partly ☐ No
Control flow graph with node–line mapping included	☒ Yes ☐ Partly ☐ No
Basis paths with test data for each path	☒ Yes ☐ Partly ☐ No
Black-box and white-box evidence compared with own results	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING (CSA37)
ASSIGNMENT 2 – CO2

CO2 Coverage: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2      return INVALID
 3  rate = 40
 4  if zone == "B" then
 5      rate = 60
 6  else if zone == "C" then
 7      rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12      w = w - 1
13  if express then
14      if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

Task 1 – Select Test Case Design Strategies **5 Marks**

CO1 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- The difference between black-box and white-box test design strategies
- The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states
- The information each strategy needs (specification, code, state model)
- The kinds of defects each strategy is likely and unlikely to reveal
- A justification for combining strategies rather than using only one

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

Ans:

Choosing a Test Design Strategy for Each Part of SwiftShip

A strategy is worth its effort only if it finds the defects that would hurt the company most. For SwiftShip those are: a wrong charge (an undercharge loses revenue on a 40,000-a-day volume, an overcharge brings refunds and complaints), a booking accepted that should have been rejected, and a parcel that moves to a state it should never reach. I choose a strategy for each part from the kind of information available and the kind of defect it can reveal.

Black-box and white-box strategies

Table 1: Black-box and white-box test design compared

Aspect	Black-box (specification-based)	White-box (structure-based)
Test basis	The required behaviour: rules S1–S4, the state model	The code or pseudocode: statements, decisions, loops, paths
Tests are chosen by	Input and output classes, boundaries, rule combinations, states	Execution of structural elements: branches, conditions, independent paths
Expected results come from	The specification (independent of the code)	The specification too; the code only decides which inputs reach which path
Strong at finding	Missing or wrong requirements handling, missing validation, wrong boundaries, wrong combinations of rules	Unreached code, untested branches, loop and condition faults, dead or unreachable code

Aspect	Black-box (specification-based)	White-box (structure-based)
Blind spot	Cannot see how the code is structured, so internal paths may stay unexecuted	Cannot find what the code does not contain (an omitted check) and cannot tell if the code does the right thing
Needs	A specification and, for tracking, a state model	Access to the code and the ability to measure coverage

Strategy selected for each part

Table 2: Selected strategies, the information they need and what they find

Part	Strategy selected	Information needed	Likely to reveal	Unlikely to reveal
Shipping-charge rules S1–S4	Equivalence classes, boundary values and a decision table (black-box)	The specification only	Wrong or missing validation (e.g. invalid zone), wrong thresholds (30, 50, 10), wrong rate or percentage, rules applied in the wrong combination	Faults visible only inside the code, e.g. a loop that misbehaves for inputs outside the chosen classes
shipCharge pseudocode	Control-flow graph, basis-path tests and branch/condition coverage (white-box)	The code (pseudocode with line numbers)	Untested branches and statements, loop mistakes, a compound condition not fully exercised, correlated or redundant decisions	Missing code: a rule that was never implemented. Also wrong expected values, because the structure says nothing about correctness
Parcel tracking	State-transition testing (black-box, model-based)	A state model: states, events, guards (retry count, cancel rule)	Illegal transitions allowed, valid transitions missing, wrong guard on the retry limit or the cancellation rule	Faults in calculation or data handling inside a state; timing and concurrency faults

Why I selected them

- **Shipping-charge rules:** the specification is short, precise and numeric, so partitions and boundaries can be derived quickly. The costly defects (wrong threshold, unvalidated zone, rule interaction) are specification-level defects. This gives the most defects for the fewest tests.
- **The pseudocode:** it contains a loop, a compound condition and an if/else-if chain. Structural tests make sure each part has actually been run. White-box design is cheap here (19 lines) and tells me what the black-box tests did not execute.
- **Tracking:** behaviour depends on the current state and history (retry count), which input-output tables do not express well. A state model makes illegal moves visible, and the 8 valid transitions can be covered with only four test paths.

Why not one strategy alone

The evidence in this assignment shows that each strategy leaves a gap that another fills. Equivalence classes alone (7 tests) found the real defect but still let 7 of the 18 seeded defects through, mostly boundary faults. Boundary tests alone (12 tests) found 14 of 18 but not the real defect, because they never varied the zone. Basis-path tests alone (9 tests) found 13 of 18 and also missed the real defect, because the defect is code that was never written. Only the combination found everything (Task 10). Combining does not cost much: after removing duplicates the combined suite shrinks to 14 tests (Task 11).

Summary of the selection: black-box techniques for rules and states (to decide what to test and what the result must be), and white-box measurement of the code (to check that nothing was left unexecuted). Random testing is added only as a cheap supplement (Task 2).

Task 2 – Evaluate Random Testing 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- How random testing would be carried out for shipCharge
- The probability that random inputs hit important values such as 30.0, 30.1 and 50.0
- The problem of deciding the expected result (test oracle) for random inputs

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

Ans:

Can Random Testing Replace Test Design?

My judgement is no. Random testing is cheap to generate and good at finding broad defects, but it cannot be relied on to hit the exact values where the specification changes behaviour, and it needs an expected-result source that a random generator does not provide.

How random testing would be carried out

I assume the generator draws, for each of 1,000 tests: weight uniformly from the one-decimal values 0.1 to 60.0 (600 values, so some inputs are invalid on purpose); zone uniformly from a list of zone codes; express with probability 0.5; and parcels uniformly from 0 to 20. The program is then run on all 1,000 inputs and each answer is compared with an expected value.

Probability of hitting the important values

With 600 equally likely weights, one random test hits a chosen weight with probability 1/600. Over 1,000 tests the chance of hitting it at least once is 1 − (599/600)^1000 = 81.1%. Table 3 gives the figures for the values the specification cares about.

Table 3: Chance that 1,000 random tests hit the important values

Value that must be tested	Calculated chance	Simulated (300 runs of 1,000 tests)
A given weight, e.g. 30.0, 30.1 or 50.0	81.1% each	76%, 82% and 79%
All three of 30.0, 30.1 and 50.0 in the same run	53% (about 0.811^3)	49%
30.0 together with Express = Yes (needed to test S3)	56.6%	57%

Value that must be tested	Calculated chance	Simulated (300 runs of 1,000 tests)
30.1 together with Express = Yes (the NOT_ALLOWED edge)	56.6%	54%
30.0 + Express + parcels = 10 (edge of S3 and S4 together)	3.9%	–
Weight exactly 0 (lower edge of S1)	0% unless the generator is told to include it	0% (not generated)

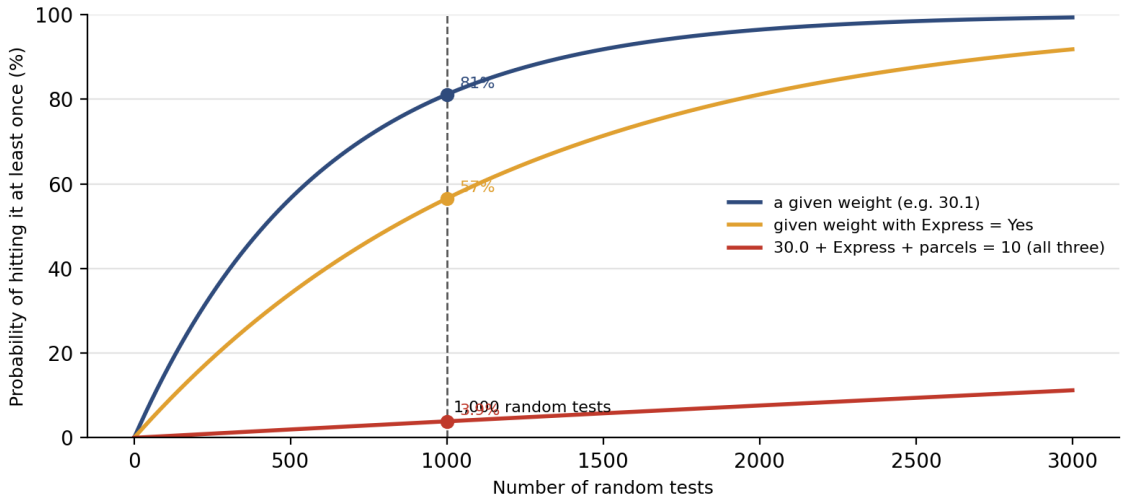


Figure 1: Chance that 1,000 random tests hit a given value at least once

So a specific edge is missed in roughly one run in five, the three key weights are all hit in only about half of the runs, and a precise combination of edges is almost never hit. Random values also land mostly in the middle of classes: only 1 value in 10 is a whole number, so "exactly 1.0 kg" and "1.1 kg" (the started-kilogram edge) are not specially favoured.

What a run of 1,000 random tests actually found

I ran the experiment against the 18 seeded defects of Task 10 (small changes to the pseudocode, such as changing > to >=), using zones A–C only. A run found on average **16.4 of 18** seeded defects, so random testing is effective against broad faults such as a wrong rate or percentage. The faults it missed are edge faults: the weight 0 mutant (never generated), the ">= 50" mutant (found in 79% of runs) and the ">= 30" mutant (found in only 57% of runs). The real defect (an invalid zone accepted) was found in **0 of 300 runs** when the zone list contained only A, B and C, because no invalid zone was ever drawn. When the generator draws zones from A–Z it is found at once, but only if someone has decided that the zone must be invalid, which is exactly the thinking that test design is.

The test-oracle problem

A random input has no expected result attached. To judge 1,000 answers there must be an oracle, and there are three possibilities. (1) Use the program itself: then every test passes and nothing is tested. (2) Write an independent reference implementation of S1–S4: this works, but it means writing the specification as code and the reference can share the same misunderstanding (for instance, if its author also forgot the invalid zone). (3) Check properties that must hold, such as: Zone B charge = 1.5 × Zone A charge for the same weight;

Express = 1.5 × non-express; discount = 0.9 × non-discount; the charge never decreases as weight grows. Properties catch rate and percentage faults without a full oracle, but they cannot say that "zone D must be INVALID".

Conclusion. Random testing cannot replace systematic design: the exact edges of S1–S4 and the oracle are the hard part, and neither is supplied by the generator. It is a useful supplement: a nightly run of 1,000 inputs with property checks costs a few seconds and guards against broad regressions between releases.

Task 3 – Design and Evaluate Equivalence Classes **6 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- Valid and invalid equivalence classes for weight, zone, express and parcels
- How you identified each class from the specification
- Test cases covering all valid classes with as few tests as possible
- Test cases covering each invalid class one at a time, and why
- Expected results calculated from the specification
- Which of your test cases would fail against the pseudocode, and what that reveals

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

Ans:

Equivalence Classes of the Shipping-Charge Rules

Classes are formed from S1–S4 only, without looking at the code. A class is a group of inputs that the specification treats in the same way, so one value represents it. The valid weight range is split at 30 kg because S3 treats weights above 30 kg differently when Express is chosen.

Table 4: Equivalence classes derived from S1–S4

Variable	Class	Description	Type	How identified in the specification
Weight	EW1	0 < weight ≤ 30	Valid	S1 gives the valid range; S3 allows Express up to 30 kg, so this part is Express-eligible
Weight	EW2	30 < weight ≤ 50	Valid	S1 valid range, but S3 returns NOT_ALLOWED when Express is chosen

Variable	Class	Description	Type	How identified in the specification
Weight	EW3	weight ≤ 0	Invalid	S1: "must be greater than 0", otherwise INVALID
Weight	EW4	weight > 50	Invalid	S1: "not more than 50", otherwise INVALID
Zone	EZ1	Zone A (₹40/kg)	Valid	S2 lists three zones, each with its own rate
Zone	EZ2	Zone B (₹60/kg)	Valid	S2
Zone	EZ3	Zone C (₹90/kg)	Valid	S2
Zone	EZ4	Any other value (D, "b", blank)	Invalid	S2: "Any other zone is INVALID"
Express	EE1	Express requested	Valid	S3 (+50%)
Express	EE2	Express not requested	Valid	S3 applies only when Express is chosen
Parcels	EP1	0 to 9 parcels this month	Valid	S4: discount only for 10 or more
Parcels	EP2	10 or more parcels this month	Valid	S4: 10% discount

Points the specification leaves open. While building the classes I found four gaps that I would raise with the analyst: (1) S1 says "up to one decimal place" but not what happens to 2.55; (2) the zone code is not said to be case-sensitive ("b"); (3) the discount rule says nothing about a negative parcel count; (4) there is no rounding rule for the final amount. I do not invent expected results for these, so they are not in the classes; they are listed as risks in Task 11.

Test cases covering the valid classes with as few tests as possible

Each valid class needs to appear in at least one test, and one test can cover one class of every variable at once. The variable with the most classes is zone (three), so at least three tests are needed, and three are enough: EC1 (EW1, EZ1, EE2, EP1), EC2 (EW1, EZ2, EE1, EP2) and EC3 (EW2, EZ3, EE2, EP1). One more test, EC4, is needed for the NOT_ALLOWED behaviour (EW2 with Express = Yes), because no "charge" test can contain it. That gives **4 tests for all valid behaviour**.

Test cases for the invalid classes, one at a time

Each invalid class gets its own test with every other input valid: EC5 (EW3), EC6 (EW4) and EC7 (EZ4), **3 tests**. The reason is error masking. Line 1 of the pseudocode returns INVALID before the zone is looked at, so a test with weight 0 **and** zone D would return INVALID whether or not the zone check worked. The invalid-zone defect would stay hidden behind the weight check. One invalid value at a time shows which check produced the answer.

Total: 7 tests represent every valid and invalid behaviour of S1–S4. The expected result of each is calculated from the specification, not from the code.

Table 5: ECP test cases, expected results and actual results of the pseudocode

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
EC1	2.5	A	No	3	EW1, EZ1, EE2, EP1	3 kg × 40 = ₹120	₹120	Pass
EC2	12.0	B	Yes	12	EW1, EZ2, EE1, EP2	12 kg × 60 = ₹720; × 1.5 = ₹1,080; × 0.9 = ₹972	₹972	Pass
EC3	40.0	C	No	0	EW2, EZ3, EE2, EP1	40 kg × 90 = ₹3,600	₹3,600	Pass
EC4	35.0	A	Yes	0	EW2 with express: NOT_ALLOWED	NOT_ALLOWED (express above 30 kg)	NOT_ALLOWED	Pass
EC5	0.0	A	No	0	Invalid weight ≤ 0	INVALID (weight out of range)	INVALID	Pass
EC6	60.0	A	No	0	Invalid weight > 50	INVALID (weight out of range)	INVALID	Pass
EC7	5.0	D	No	0	Invalid zone	INVALID (zone not A, B or C)	₹200	FAIL

Which tests fail against the pseudocode, and what it reveals

Six of the seven tests pass. **EC7 fails**: weight 5.0 kg to zone D should be INVALID (S2), but the pseudocode returns ₹200, which is 5 kg × ₹40, the Zone A price. It reveals that the code has no branch for an invalid zone: the if / else-if chain on lines 4–7 only has "B" and "C", and rate keeps its initial value 40 for any other value. The effect is real money: a customer who mistypes Zone "b" for Zone B is charged ₹40 instead of ₹60 per kilogram, an undercharge of 33%, and the booking is accepted without any warning. I confirmed by running the code on a grid of 16,000 weight/zone/express/parcel combinations that this is the only point at which the pseudocode differs from S1–S4.

How effective are the classes?

- **Strong:** 7 tests, 1 real defect found, and the set also reaches 100% statement, branch and condition coverage of the pseudocode (19 of 19 statements, 14 of 14 branches), which is more than a strictly structural approach would promise for 7 tests.
- **Weak at the edges:** when I seeded 18 small faults into the code, the ECP tests found 11 of 18. Six of the seven they missed are threshold faults (> 50 written as >= 50, > 30 written as >= 30, >= 10 written as > 10 and so on), because the test values 2.5, 12, 40, 60, 5 are comfortably inside their classes; a value inside a class cannot tell where the class ends. The seventh is a combination fault (Task 5).
- **Weak at combinations:** EC1–EC3 never use a discount without Express, or Express without a discount. A fault that applies the discount only to Express parcels survives (Task 5).

So equivalence classes are the right first step and found the one specification-level defect, but they must be followed by boundary tests (Task 4) and rule combinations (Task 5).

Task 4 – Design and Evaluate Boundary Value Tests **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)
- Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces
- Test cases with expected results for the boundaries you select
- Which form of BVA you recommend for shipCharge, and why

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

Ans:

Boundary Values of SwiftShip

The boundaries most likely to hide defects are those where the code compares a number with a constant, because one wrong operator (> instead of >=) moves the edge by one value. In SwiftShip these are: weight 0 | 0.1 (line 1), weight 50 | 50.1 (line 1), weight 30 | 30.1 (line 14, Express only), parcels 9 | 10 (line 17) and, less obviously, the started-kilogram edge: 1.0 kg is one kilogram but 1.1 kg is two (line 10, w > 0), which repeats at every whole number.

Three forms of BVA and what they cost

For n numeric variables, normal BVA tests minimum, just above minimum, nominal, just below maximum and maximum of one variable at a time (4n + 1 tests); robust BVA adds one value just outside each end (6n + 1); worst-case BVA takes every combination of the five values (5^n); robust worst-case takes every combination of seven (7^n). SwiftShip has n = 2 numeric variables, weight and parcels. Zone and Express are categories, so each multiplies the count again if it is included.

Table 6: Forms of BVA for weight and parcels, and the seeded defects each found

Form of BVA	Formula	Tests (n = 2)	Seeded defects found (of 18)	Comment
Normal BVA (range extremes)	4n + 1	9	6	Tests only the ends of the ranges; never tests 30 or 10
Robust BVA (range extremes)	6n + 1	13	9	Adds 0.0 and 50.1, still misses 30 | 30.1 and 9 | 10

Form of BVA	Formula	Tests (n = 2)	Seeded defects found (of 18)	Comment
Worst-case BVA (range extremes)	5^n	25	6	All combinations of the same extremes: more tests, no new edge
Robust worst-case BVA	7^n	49	not run	Would multiply to 49 × 3 zones × 2 = 294 with the other variables
Boundary values taken from the rules (my choice)	one edge at a time	12	14	Tests every threshold of S1, S3 and S4 and the started-kilogram rule

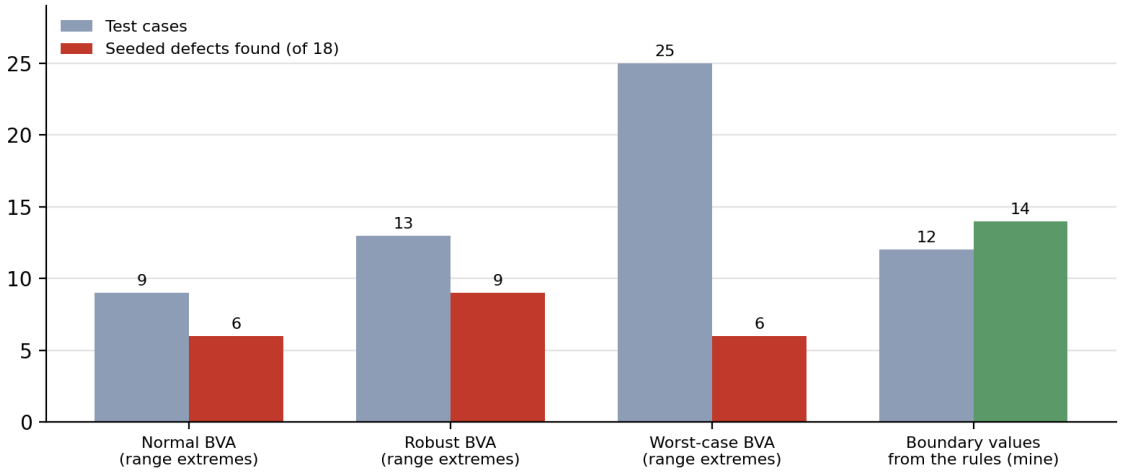


Figure 2: Number of tests and seeded defects found by each form of BVA

The textbook forms take the "minimum" and "maximum" of each variable. In SwiftShip the dangerous edges are **internal thresholds** (30 kg and 10 parcels) that are not range extremes, so those forms never touch them, and with Express = No the 30 kg threshold cannot be seen at all. They found only 6 to 9 of the 18 seeded defects, and the 25 worst-case tests found no more than the 9 normal ones (6 of 18). More combinations of the same wrong values add cost, not information.

Table 7: Boundary value test cases (zone A unless the edge needs Express) with expected results

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
BV1	0.0	A	No	0	Lower invalid edge (S1)	INVALID (weight out of range)	INVALID	Pass
BV2	0.1	A	No	0	Smallest valid weight, 1 kg	1 kg × 40 = ₹40	₹40	Pass
BV3	1.0	A	No	0	Exactly 1 kg is 1 kg	1 kg × 40 = ₹40	₹40	Pass
BV4	1.1	A	No	0	Started kilogram: 1.1 kg is 2 kg	2 kg × 40 = ₹80	₹80	Pass
BV5	30.0	A	Yes	0	Express edge: 30.0 allowed	30 kg × 40 = ₹1,200; × 1.5 = ₹1,800	₹1,800	Pass

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
BV6	30.1	A	Yes	0	Express edge: 30.1 NOT_ALLOWED	NOT_ALLOWED (express above 30 kg)	NOT_ALLOWED	Pass
BV7	30.1	A	No	0	Edge without Express: 31 kg charged	31 kg × 40 = ₹1,240	₹1,240	Pass
BV8	49.9	A	No	0	Just below the maximum (50 kg)	50 kg × 40 = ₹2,000	₹2,000	Pass
BV9	50.0	A	No	0	Maximum valid weight	50 kg × 40 = ₹2,000	₹2,000	Pass
BV10	50.1	A	No	0	Upper invalid edge (S1)	INVALID (weight out of range)	INVALID	Pass
BV11	10.0	A	No	9	9 parcels: no discount	10 kg × 40 = ₹400	₹400	Pass
BV12	10.0	A	No	10	10 parcels: discount starts	10 kg × 40 = ₹400; × 0.9 = ₹360	₹360	Pass

All twelve tests pass the pseudocode: the code handles every numeric edge correctly. This is a useful result in itself, since these are the places where defects are most likely. The boundary tests all use Zone A, so they could not find the invalid-zone defect; BVA applies to ordered numbers, not to categories like the zone.

Recommended form

I recommend **boundary values taken from the rules, one edge at a time: 12 tests**. It is a robust form (it includes the invalid side of each edge) but it is aimed at the actual thresholds in S1, S3 and S4 and the started-kilogram rule instead of the range extremes. It finds 14 of the 18 seeded defects with 12 tests, against 9 of 18 for robust BVA with 13 tests, and 6 of 18 for worst-case with 25. Its risk-based reasons: each threshold decides a charge or a rejection for thousands of parcels a day, whereas combining weight and parcels values adds no risk because the discount is applied independently of the weight. The interaction that does matter (Express with discount) is handled by the decision table in Task 5, not by multiplying boundary values.

Task 5 – Apply Other Black-Box Techniques **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- A cause–effect graph or decision table for rules S3 and S4
- Test cases derived from the decision table
- Combinations that ECP/BVA did not cover, and whether they matter

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

Ans:

Decision Table for the Express, Weight and Discount Rules

S3 and S4 are rules that act together: Express changes the charge and may refuse the booking, and the discount applies to whatever the charge has become. Four conditions decide the outcome: C1 the weight is valid (S1), C2 Express is requested, C3 the weight is 30 kg or less, C4 the customer has 10 or more parcels. With four yes/no conditions there are 16 combinations; don't-care entries reduce them to six rules. (The zone is left out because S2 acts on the base charge only and is covered by ECP.)

Table 8: Decision table for S1, S3 and S4

Condition / action	R1	R2	R3	R4	R5	R6
C1 Weight valid (0 < w ≤ 50)?	N	Y	Y	Y	Y	Y
C2 Express requested?	–	Y	Y	Y	N	N
C3 Weight ≤ 30 kg?	–	N	Y	Y	–	–
C4 Parcels this month ≥ 10?	–	–	N	Y	N	Y
A1 Return INVALID	✓					
A2 Return NOT_ALLOWED		✓				
A3 Charge = base (kg × rate)			✓	✓	✓	✓
A4 Add 50% for Express			✓	✓		
A5 Apply 10% discount				✓		✓

Test cases derived from the decision table (one per rule)

Table 9: Decision-table test cases

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
DT1	0.0	A	No	0	R1: invalid weight	INVALID (weight out of range)	INVALID	Pass
DT2	35.0	C	Yes	12	R2: Express above 30 kg, with discount parcels	NOT_ALLOWED (express above 30 kg)	NOT_ALLOWED	Pass
DT3	4.0	A	Yes	5	R3: Express, no discount	4 kg × 40 = ₹160; × 1.5 = ₹240	₹240	Pass
DT4	4.0	B	Yes	10	R4: Express and discount	4 kg × 60 = ₹240; × 1.5 = ₹360; × 0.9 = ₹324	₹324	Pass

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
DT5	4.0	C	No	5	R5: no Express, no discount	4 kg × 90 = ₹360	₹360	Pass
DT6	4.0	A	No	10	R6: discount without Express	4 kg × 40 = ₹160; × 0.9 = ₹144	₹144	Pass

DT2 deserves a remark: the weight is above 30 kg with Express, and the customer also has 12 parcels. S3 says the request is NOT_ALLOWED, and no discount should appear. This tests that the refusal comes before any money is calculated. The zones in the six tests rotate (A, C, A, B, C, A) so that no zone is left out.

Comparison with the ECP and BVA tests

Table 10: Rules covered by each test set, and the combination faults each finds

Test set	R1	R2	R3	R4	R5	R6	Rules covered	Finds "discount only for Express" / "discount only without Express"
ECP (7 tests)	✓	✓	–	✓	✓	–	4 of 6	missed / found
BVA (12 tests)	✓	✓	✓	–	✓	✓	5 of 6	found / missed
Decision table (6 tests)	✓	✓	✓	✓	✓	✓	6 of 6	found / found

Do ECP and BVA miss combination defects? Yes, and they miss different ones. ECP covers 4 of the 6 rules: it never has Express without a discount (R3) or a discount without Express (R6). BVA covers 5 of 6 but never has Express together with a discount (R4). To check the effect I made two seeded faults a developer could plausibly introduce when tidying the code: the discount moved inside the Express block (discount only for Express), and the discount made an “else” of the Express test (discount only without Express). The ECP tests found the second but missed the first; the BVA tests found the first but missed the second; the six decision-table tests found both. A suite built from either technique alone would have passed a release in which one of the two groups of customers (Express or non-Express) lost its discount, a revenue-and-complaints defect that is invisible in single-value tests.

Combinations still not covered, and whether they matter

- **Zone × rule:** each decision-table test uses one zone, so a given zone is combined with only two or three of the six rules. In the present code the zone only fixes the rate before Express and discount are applied, so the risk is small. It would not be small if a rule depended on the zone, and in Task 12 the new surcharge does depend on Zone C.
- **Weight × rule:** the rules are tested at 4 kg. Large weights with a discount (for example 49.9 kg with 10 parcels) are not tested together. The discount is a multiplication that does not depend on weight, so I rate this low.
- **NOT_ALLOWED × discount:** covered by DT2.

Verdict: the decision table adds only 6 tests and closes a real blind spot of ECP and BVA, so it is worth its cost. Full 16-way coverage of C1–C4 adds nothing, since the don’t-care entries are rules that the specification does not distinguish.

Task 6 – Design and Evaluate State-Transition Tests **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- A state-transition diagram and table for parcel tracking
- Test cases for all valid transitions and selected invalid transitions
- Tests for the "maximum two retries" rule and the cancellation rule

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

Ans:

State-Transition Model of Parcel Tracking

The model has eight states and eight valid transitions. Two rules depend on history or position, not just on the current state: the retry count (maximum two retries) and the position of cancellation (only before PICKED_UP). I read "maximum two retries" as: the first delivery attempt plus up to two retries, so a parcel can fail three times; after the third failure the only legal move is RETURNED. The specification does not say this explicitly, so I list it as an assumption to be confirmed.

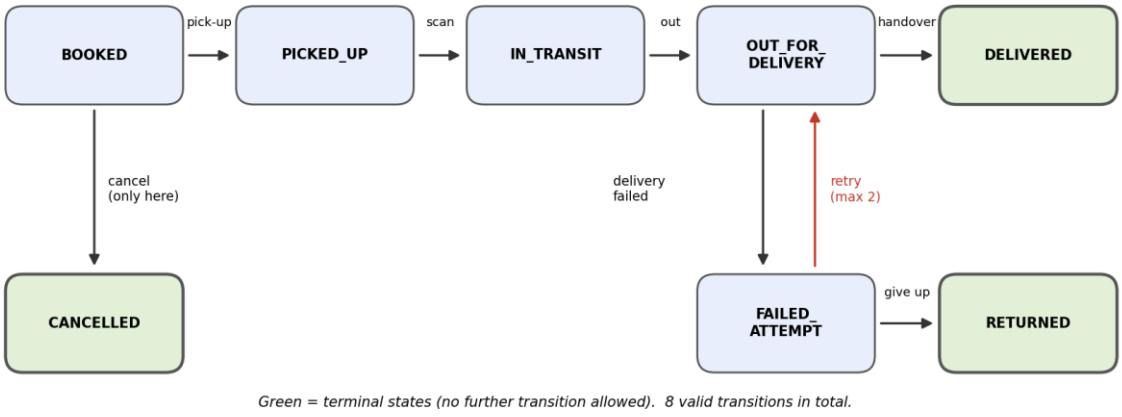


Figure 3: State-transition diagram of parcel tracking

Table 11: State-transition table (valid transitions)

From state	To state (event)	Guard / rule
BOOKED	PICKED_UP (pick-up scan)	–
BOOKED	CANCELLED (customer cancels)	Cancellation allowed only in this state

From state	To state (event)	Guard / rule
PICKED_UP	IN_TRANSIT (hub scan)	–
IN_TRANSIT	OUT_FOR_DELIVERY (out scan)	–
OUT_FOR_DELIVERY	DELIVERED (handover)	–
OUT_FOR_DELIVERY	FAILED_ATTEMPT (customer absent)	–
FAILED_ATTEMPT	OUT_FOR_DELIVERY (retry)	Only if fewer than 2 retries have been made
FAILED_ATTEMPT	RETURNED (give up)	Always possible; the only move after the third failed attempt
DELIVERED, RETURNED, CANCELLED	none	Terminal states

With 8 states there are 8 × 8 = 64 ordered pairs, including a state to itself; 8 are valid and **56 are invalid**. Testing every invalid pair would be 56 tests; I select by risk (below).

Tests for all valid transitions

Four test paths cover all eight valid transitions (0-switch coverage):

Table 12: Valid transition tests

ID	Path (events in order)	Transitions covered
VT1	BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED	The normal path: 4 transitions
VT2	BOOKED → CANCELLED	Cancellation before pick-up
VT3	… OUT_FOR_DELIVERY → FAILED_ATTEMPT → RETURNED	Failure and return: 2 transitions
VT4	… OUT_FOR_DELIVERY → FAILED_ATTEMPT → OUT_FOR_DELIVERY → DELIVERED	Retry that succeeds: 1 new transition

Tests for the retry rule and the cancellation rule

Table 13: Retry and cancellation tests (the rules sit on the edges of their allowed ranges)

ID	Test	Expected result
RT1	Fail, retry 1, fail, retry 2, deliver (second retry succeeds)	Accepted: exactly two retries are allowed
RT2	Fail, retry 1, fail, retry 2, fail, then try a third retry	Third retry rejected; only RETURNED is offered
RT3	After the third failure, move to RETURNED	Accepted
CT1	Cancel when BOOKED (= VT2)	Accepted
CT2	Cancel when PICKED_UP (first state after the allowed range)	Rejected; the parcel is already with the company

ID	Test	Expected result
CT3	Cancel when OUT_FOR_DELIVERY	Rejected; the parcel is on the van
CT4	Cancel when DELIVERED	Rejected; a delivered parcel cannot be cancelled and refunded

Invalid transitions selected, and why

I choose invalid transitions by what the company would lose if the system allowed them. Transitions that go backwards or skip a step are handled by the same check in the code, so one representative of each kind is enough; terminal states and money-related moves are tested separately.

Table 14: Selected invalid transitions and the loss each could cause

ID	Invalid transition	Why it is selected (money or reputation)
IV1	BOOKED → DELIVERED	A parcel never picked up would show as delivered: false proof of delivery, refund disputes and lost-parcel claims
IV2	IN_TRANSIT → DELIVERED	Skips the out-for-delivery scan: delivery with no last-mile record, the usual route for fraudulent "not received" claims
IV3	DELIVERED → OUT_FOR_DELIVERY	A delivered parcel re-opened: duplicate delivery cost and a confusing status for the customer
IV4	DELIVERED → RETURNED	Delivered parcel changed to returned: invites refund claims after the customer has the goods
IV5	CANCELLED → PICKED_UP	A refunded booking is collected anyway: the company carries the cost without a charge
IV6	RETURNED → OUT_FOR_DELIVERY	A parcel returned to the sender is sent out again: wrong delivery and double handling
IV7	FAILED_ATTEMPT → DELIVERED	Delivered without a delivery attempt: false proof of delivery after a failure
IV8	OUT_FOR_DELIVERY → PICKED_UP	A backward move: stands for all backward moves, which would corrupt the tracking history shown to customers

Together with RT2 and CT2–CT4 this gives 12 invalid cases out of 56 (the other 44 follow the same rules; I rate the risk of each of them lower). The tracking suite therefore has 4 + 3 + 3 + 8 = **18 tests** (VT1–VT4, RT1–RT3, CT2–CT4, IV1–IV8).

Which invalid transitions are most costly? In my judgement IV1, IV2 and IV7 (false delivery) and CT4 (cancellation after delivery) cost the most, because they can turn into refund or compensation claims against the company, and IV5 (collecting a cancelled booking) because the work is done without being paid.

Task 7 – Construct the Control Flow Graph **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- Nodes and edges, with line numbers mapped to nodes
- Decision nodes, the loop, and the compound condition at line 1
- Differences between the code structure and the rules S1–S4 (for example, invalid zones)

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

Ans:

Control Flow Graph of shipCharge

A node is a block of statements that always run together; a decision node ends with a condition that has two outcomes. I merged straight-line statements (lines 8–9, lines 11–12) into one node each. Table 15 gives the mapping, Figure 4 the graph.

Table 15: Mapping of pseudocode lines to graph nodes

Node	Line(s)	Statement	Kind
N1	1	if weight <= 0 or weight > 50	Decision (compound condition)
N2	2	return INVALID	Exit
N3	3	rate = 40	Statement
N4	4	if zone == "B"	Decision
N5	5	rate = 60	Statement
N6	6	else if zone == "C"	Decision
N7	7	rate = 90	Statement
N8	8–9	charge = 0; w = weight	Statement block (join of zone branches)
N9	10	while w > 0	Decision (loop test)
N10	11–12	charge = charge + rate; w = w - 1	Loop body
N11	13	if express	Decision
N12	14	if weight > 30	Decision
N13	15	return NOT_ALLOWED	Exit
N14	16	charge = charge * 1.5	Statement
N15	17	if parcels >= 10	Decision
N16	18	charge = charge * 0.9	Statement

Node	Line(s)	Statement	Kind
N17	19	return charge	Exit

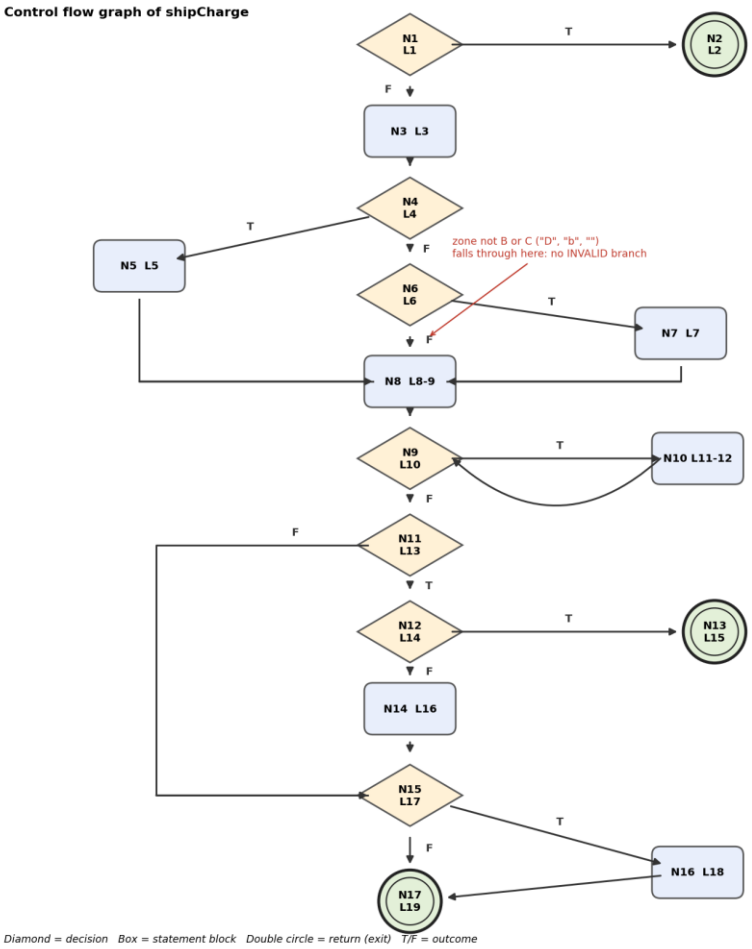


Figure 4: Control flow graph of shipCharge (nodes N1–N17, lines L1–L19)

Table 16: Edges of the graph (T/F = outcome of the decision)

Edge	From → To	Edge	From → To
e1	N1 → N2 (T)	e13	N10 → N9
e2	N1 → N3 (F)	e14	N11 → N12 (T)
e3	N3 → N4	e15	N11 → N15 (F)
e4	N4 → N5 (T)	e16	N12 → N13 (T)
e5	N4 → N6 (F)	e17	N12 → N14 (F)
e6	N5 → N8	e18	N14 → N15
e7	N6 → N7 (T)	e19	N15 → N16 (T)
e8	N6 → N8 (F)	e20	N15 → N17 (F)
e9	N7 → N8	e21	N16 → N17

Edge	From → To	Edge	From → To
e10	N8 → N9	e22–e24	N2, N13, N17 → EXIT (virtual)
e11	N9 → N10 (T)		
e12	N9 → N11 (F)		

The graph has 17 nodes plus one virtual EXIT node (the three returns are joined to it so that the formulas for complexity apply) and 24 edges.

Decision nodes, the loop and the compound condition

- **Decision nodes:** seven: N1 (line 1), N4 (4), N6 (6), N9 (10), N11 (13), N12 (14) and N15 (17). Each has two outgoing edges.
- **The loop:** N9 tests w > 0, N10 is the body and the edge N10 → N9 is the back edge. The only way out of the loop is the F edge of N9. Each pass lowers w by 1, so the number of passes is the number of started kilograms (verified for all 500 weights from 0.1 to 50.0: passes = the weight rounded up).
- **The compound condition at line 1:** "weight <= 0 or weight > 50" is one decision in the statement-level graph above, but it is two conditions. With short-circuit evaluation, N1 splits into N1a (weight <= 0) and N1b (weight > 50): if N1a is true the program returns INVALID, otherwise N1b is evaluated. This adds one node and two edges (19 nodes including EXIT, 26 edges), and is needed for condition coverage in Task 9.

Does the code structure match the specification?

Table 17: Specification rules compared with the code structure

Rule	What the specification says	What the code does	Match?
S1	Reject weight ≤ 0 or > 50 as INVALID	N1 returns INVALID for either condition	Matches
S2 (rates)	Three zones, three rates; charge for each started kilogram	Rate chain N3–N7, loop N9–N10 adds the rate once per kilogram	Matches, but the loop is a different structure from a one-line rounding-up formula
S2 (invalid zone)	Any other zone is INVALID	No decision or return for it: the F edge of N6 (e8) joins straight to N8, so any other value is priced at ₹40	**Mismatch: the specification has four outcomes for the zone, the code only three**
S3	Express +50%; above 30 kg NOT_ALLOWED	N11–N14 do the same, but only after the whole loop has already been run	Matches in result; the check could come before the loop
S4	10% off the final charge if 10 or more parcels	N15–N16 apply it last, after Express	Matches
Not in the specification	Rounding of the amount; more than one decimal in weight	No rounding and no check of the decimal places	Gap in the specification, not tested by the code

Evaluation. The graph shows the one defect that matters directly: the zone chain has no branch for an invalid zone. A structural view cannot tell this, because every node that exists is correct; what is wrong is a node that is missing. Only comparison with S2 shows it. The other differences (the loop instead of a formula, the late Express check) cost time or readability but not correctness. On the loop: a weight with floating-point error could in theory lead to one pass too many, but for the one-decimal weights allowed by S1 the loop count is exact.

Task 8 – Evaluate Path Testing and Cyclomatic Complexity **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- Cyclomatic complexity calculated by at least two methods
- A set of independent (basis) paths and test data for each path
- Infeasible paths, if any, and why
- How the loop affects path testing, and how you would test it (zero, one, many iterations)

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

Ans:

Cyclomatic Complexity and Basis Paths

Cyclomatic complexity by three methods

The statement-level graph of Task 7 has N = 18 nodes (including the virtual EXIT), E = 24 edges and one connected piece (P = 1).

- **Edges and nodes:** V(G) = E − N + 2P = 24 − 18 + 2 = **8**.
- **Decision nodes:** V(G) = (number of decision nodes) + 1 = 7 + 1 = **8** (N1, N4, N6, N9, N11, N12, N15).
- **Regions:** the graph drawn without crossing lines has 7 enclosed regions (the loop, the zone branches, the Express block and the discount branch, among others) plus the outside region, 8 in all; V(G) = 8.

If the compound condition on line 1 is counted as two conditions (N1a, N1b of Task 7), E = 26, N = 19 and V(G) = 26 − 19 + 2 = **9**, or 8 + 1 = 9 by counting conditions. I use 8 for the independent-path set and 9 for the condition-level view (Task 9). V(G) = 8 means at least 8 tests are needed to execute every independent path once.

Basis paths and test data

I took the path with zone A, no Express, no discount and one pass through the loop (BP2) as

the baseline and flipped one decision at a time. Flipping the loop decision gives the path that skips the loop; this path is infeasible (see below), so I replaced it with a path through the loop several times (BP8), which is independent of the others because it carries the back edge more than once.

Table 18: Basis paths, test data and results

Path	Nodes visited	Decisions taken (outcome)
BP1	N1, N2, EXIT	N1 true
BP2	N1, N3, N4, N6, N8, N9, N10, N9, N11, N15, N17	N1 F, N4 F, N6 F, N9 T then F, N11 F, N15 F
BP3	N1, N3, N4, N5, N8, N9, N10, N9, N11, N15, N17	as BP2 but N4 true (zone B)
BP4	N1, N3, N4, N6, N7, N8, N9, N10, N9, N11, N15, N17	as BP2 but N6 true (zone C)
BP5	… N11, N12, N14, N15, N17	as BP2 but N11 true, N12 false (Express, 30 kg or less)
BP6	… N11, N12, N13, EXIT	as BP2 but N11 true, N12 true (Express above 30 kg)
BP7	… N15, N16, N17	as BP2 but N15 true (10 or more parcels)
BP8	… N9, N10, N9, N10, N9, N10, N9 …	as BP2 but the loop runs three times

Table 19: Basis path test cases, expected results and actual results

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
BP1	0.0	A	No	0	Path 1: invalid weight	INVALID (weight out of range)	INVALID	Pass
BP2	1.0	A	No	0	Path 2 (baseline): zone A, no Express, no discount, 1 loop pass	1 kg × 40 = ₹40	₹40	Pass
BP3	1.0	B	No	0	Path 3: zone B	1 kg × 60 = ₹60	₹60	Pass
BP4	1.0	C	No	0	Path 4: zone C	1 kg × 90 = ₹90	₹90	Pass
BP5	1.0	A	Yes	0	Path 5: Express up to 30 kg	1 kg × 40 = ₹40; × 1.5 = ₹60	₹60	Pass
BP6	35.0	A	Yes	0	Path 6: Express above 30 kg	NOT_ALLOWED (express above 30 kg)	NOT_ALLOWED	Pass
BP7	1.0	A	No	10	Path 7: discount	1 kg × 40 = ₹40; × 0.9 = ₹36	₹36	Pass
BP8	2.5	A	No	0	Path 8: loop run several times	3 kg × 40 = ₹120	₹120	Pass

All eight tests pass: on each structural path the pseudocode computes what the specification requires. That is exactly the limit of path testing: it shows that each path works for the data chosen, not that the right paths exist.

Infeasible paths

- **Zero loop passes:** N8 → N9 (F) → N11 would skip the loop. Line 1 has already rejected every weight ≤ 0, so on reaching the loop w = weight > 0 and N9 is true the first time. This path cannot be run.
- **Loop passes beyond 50:** a weight above 50 is rejected at line 1, so 51 or more passes are impossible.
- **Zone B and zone C together:** impossible, since the decisions are exclusive.
- **Express above 30 kg together with the 1.5 factor:** the code returns before it (N13), so N14 never follows N12 true.

How the loop affects path testing

The loop makes the number of paths depend on the data: each passes count from 1 to 50 is a different path. Counting the feasible paths exactly: for each of the three zone routes (B, C, other), weights that need 1–30 passes allow 4 outcomes (Express or not, discount or not) and weights needing 31–50 passes allow 3 (no Express with or without discount, or NOT_ALLOWED). That is 30 × 4 + 20 × 3 = 180 paths per zone route, × 3 routes = 540, plus the single INVALID path: **541 feasible paths**. Testing all of them is not sensible, so I use loop testing, taking the iteration count at its boundaries:

Table 20: Loop tests for the loop on line 10 (zone A, no Express, no discount)

Iterations	Feasible?	Test
0	No (infeasible, see above)	–
1	Yes	LP1: weight 0.1
2	Yes	LP2: weight 1.1
Many (typical)	Yes	LP3: weight 12.0
Maximum − 1 = 49	Yes	LP4: weight 49.0
Maximum = 50	Yes	LP5: weight 50.0
Maximum + 1 = 51	No: weight above 50 is rejected on line 1	BV10 (weight 50.1) shows the rejection

ID	Weight	Zone	Express	Parcels	Purpose	Expected result (from S1–S4)	Actual	Result
LP1	0.1	A	No	0	1 pass (smallest weight)	1 kg × 40 = ₹40	₹40	Pass
LP2	1.1	A	No	0	2 passes (1 kg + 0.1 kg)	2 kg × 40 = ₹80	₹80	Pass
LP3	12.0	A	No	0	Many passes (12)	12 kg × 40 = ₹480	₹480	Pass
LP4	49.0	A	No	0	Max − 1 passes (49)	49 kg × 40 = ₹1,960	₹1,960	Pass
LP5	50.0	A	No	0	Maximum passes (50)	50 kg × 40 = ₹2,000	₹2,000	Pass

Is the number of tests enough?

V(G) = 8 is the minimum number of tests that exercise all independent paths; **it is not enough** to test the logic of shipCharge, for four reasons. (1) The 8 paths are 8 of 541 feasible ones, and each is run with one convenient input: the Express path was run at 1.0 kg, so the edge at 30 kg is never visited. Measured on my seeded defects, the basis tests (with the one extra test of Task 9) found only 13 of 18 and missed the faults at exactly 50 kg, at 30 kg (both versions) and at 9 parcels. (2) The basis set cannot detect missing code: all eight pass although the code accepts an invalid zone. (3) The paths say nothing about whether the answer is right unless an expected result from the specification is used. (4) The loop needs its own tests (Table 20). The number of tests for structural coverage is therefore 8 + 1 (the compound condition, Task 9) + 5 loop tests, and the logic is only adequately tested in combination with the black-box tests.

Task 9 – Judge Test Adequacy and Coverage **3 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- Statement, branch, condition and multiple-condition coverage for shipCharge
- The coverage achieved by your path tests from Task 8
- The additional tests needed to reach the next stronger criterion
- Whether 100 % coverage would have found the invalid-zone defect

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

Ans:

Coverage Criteria and Adequacy for shipCharge

Table 21: Coverage criteria and the number of items in shipCharge

Criterion	Meaning	Items in shipCharge
Statement	Every statement runs at least once	19 statements (lines 1–19)
Branch (decision)	Every decision is true once and false once	7 decisions × 2 = 14 branches
Condition	Every atomic condition is true once and false once	16 outcomes: line 1 has 2 conditions (4 outcomes), the 6 other decisions have 1 each (12)

Criterion	Meaning	Items in shipCharge
Multiple condition	Every combination of condition outcomes	Only line 1 is compound: (T,–), (F,T), (F,F) are feasible; (T,T) is impossible, so 3 combinations
Path	Every path from start to end	541 feasible paths (Task 8)

Coverage achieved by my tests

Table 22: Coverage reached by each test set (measured by running instrumented pseudocode)

Test set	Tests	Statement (of 19)	Branch (of 14)	Condition (of 16)	Multiple condition, line 1 (of 3)
Basis paths BP1–BP8 (Task 8)	8	19 / 19	14 / 14	15 / 16	2 / 3
Basis paths + BP9 (weight 50.1)	9	19 / 19	14 / 14	16 / 16	3 / 3
ECP tests (Task 3)	7	19 / 19	14 / 14	16 / 16	3 / 3
BVA tests (Task 4)	12	17 / 19	12 / 14	14 / 16	3 / 3
Decision-table tests (Task 5)	6	19 / 19	14 / 14	15 / 16	2 / 3
Final suite (Task 11)	14	19 / 19	14 / 14	16 / 16	3 / 3

The path tests of Task 8 reach 100% statement and branch coverage (19 of 19 and 14 of 14), but **15 of 16 condition outcomes**, 93.8%. The missing outcome is "weight > 50 is true": BP1 uses weight 0, which makes the first condition true so the second is never evaluated (short-circuit). One more test, **BP9 with weight 50.1** (expected INVALID, actual INVALID), reaches 16 of 16 and all 3 combinations of line 1. So the step from branch to condition / multiple-condition coverage costs exactly one test, because line 1 is the only compound condition.

How many tests each criterion needs at minimum

Found by solving the smallest covering set over the candidate inputs: **statement coverage 3 tests, branch coverage 4 tests, condition coverage 5 tests** (the branch tests plus one with weight above 50). Path coverage would need 541. These small numbers show that coverage criteria are cheap to meet and therefore cannot by themselves prove that the testing is good.

Would 100% coverage have found the invalid-zone defect?

No. The defect is a missing branch: the code has no statement for an invalid zone, so there is nothing for a coverage tool to report as unexecuted. An invalid zone such as "D" takes the same path as Zone A (N4 false, N6 false), which is already covered by any Zone A test; adding it raises no coverage figure. I demonstrate it with four tests that give 100% statement and branch coverage (19 of 19 and 14 of 14) and all four pass against the specification:

Table 23: A coverage-complete set that does not find the defect

ID	Weight	Zone	Express	Parcels	Expected result	Actual	Result
CV1	0.0	A	No	0	INVALID	INVALID	Pass
CV2	10.0	B	Yes	10	₹810	₹810	Pass
CV3	1.0	C	No	0	₹90	₹90	Pass
CV4	35.0	A	Yes	0	NOT_ALLOWED	NOT_ALLOWED	Pass

Coverage therefore shows what was **executed**, never what was **specified**. Even all 541 paths would not have exposed the defect, as no path represents it.

Recommended adequacy criterion

For a module that prices 40,000 parcels a day I recommend, as the entry condition for a release, **100% branch coverage and 100% condition coverage (including all three combinations of line 1), together with 100% coverage of the specification: all equivalence classes, all boundary values and all decision-table rules.** Reasons:

- Statement coverage (3 tests) is too weak: it does not force every decision to take its false outcome.
- Branch plus condition coverage costs 5 tests in this module and exposes the compound condition, where a one-word slip (and for or) is a plausible fault.
- Full multiple-condition or path coverage adds nothing here (3 combinations for one line, 541 paths) and cannot be afforded.
- The specification criteria are what find the invalid-zone defect, so they must be part of the gate. A single coverage number is a necessary but not a sufficient measure of adequacy; management should be given the three numbers (code coverage, specification coverage, defects found), not one.

Task 10 – Compare Black-Box and White-Box Effectiveness **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- The defects found by black-box tests only, white-box tests only, and both
- Reasons why each approach missed what it missed
- Overlapping tests that can be removed without losing defect-finding power
- Whether this result would hold for other modules

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

Ans:

Black-Box against White-Box: What Each Found

The code has one real defect against S1–S4 (D1: an invalid zone is accepted and priced as Zone A). Since a real defect count of one says little, I added 18 plausible small faults to copies of the pseudocode (for example > changed to >=, a rate changed, a discount moved) and ran every test set against them. A seeded fault is "found" when a test gives a different result from the correct program. The 18 are standing for the faults a later change could introduce; they are my own and cannot replace field data.

Table 24: Defects found by black-box tests only, white-box tests only and both

Group	Count	Defects
Black-box only	1 real + 5 seeded	D1 invalid zone accepted (found by ECP test EC7 only); M2 weight >= 50; M11 weight >= 30 (Express); M12 weight > 31 (Express); M15 parcels >= 9; M18 discount only without Express
White-box only	0	None
Both	13 seeded	M1, M3, M4, M5, M6, M7, M8, M9, M10, M13, M14, M16, M17 (wrong thresholds on the weight range, rates, loop faults, factors)
Neither	0	None among the 18; the real defect was found

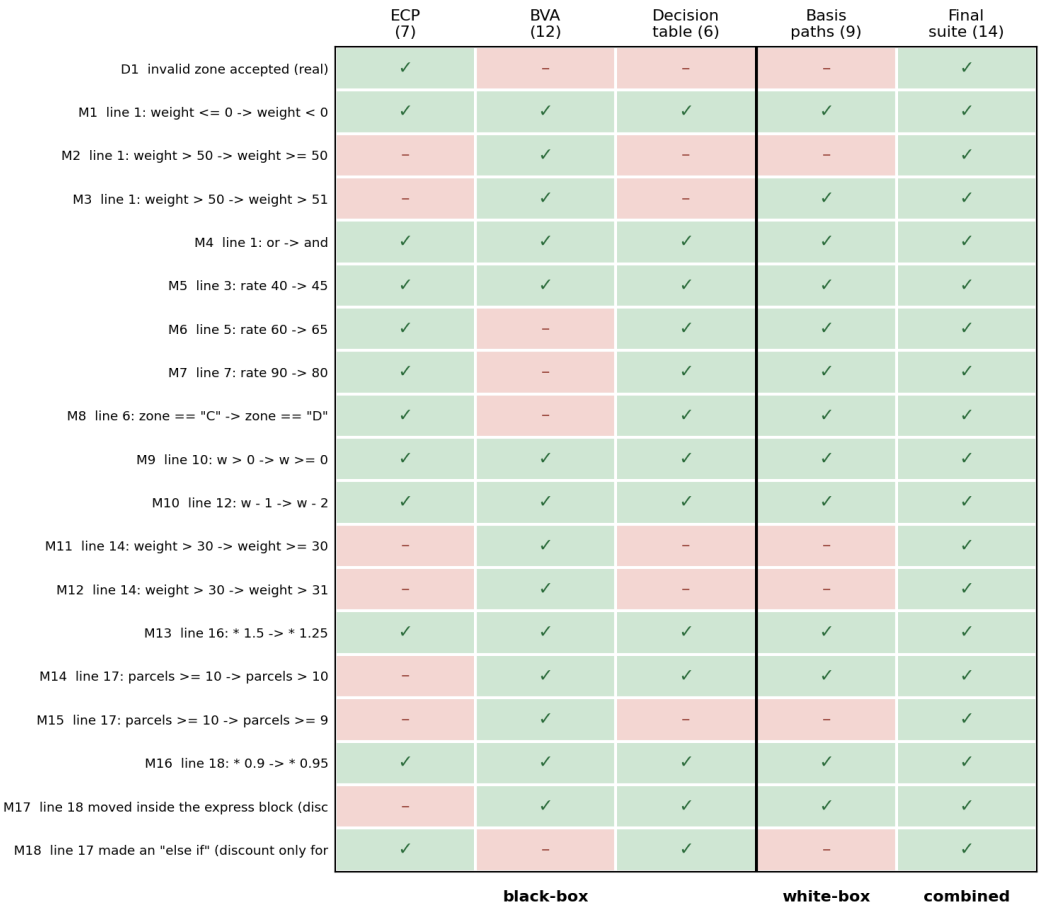


Figure 5: Which technique found which defect

Why each approach missed what it missed

- **White-box (basis paths) missed D1** because the defect is an omission. A path test only chooses data for paths that exist, and the invalid zone has no path of its own.
- **White-box missed M2, M11, M12 and M15** (edges at 50 kg, at 30 kg, at 9 parcels): to execute a path any value that satisfies its conditions will do, so I took convenient values (1.0 kg, 35 kg). The structure does not say that 30.0 and 30.1 are the interesting ones.
- **White-box missed M18** (discount only without Express): basis paths change one decision at a time from the baseline, so none of them has Express and a discount together.
- **ECP missed seven (M2, M3, M11, M12, M14, M15, M17):** test values inside the classes cannot show where a class ends, and no ECP test has a discount without Express.
- **BVA missed M6, M7, M8 and M18:** all BVA tests used Zone A and almost no Express with discount; boundaries on numbers say nothing about zone rates or the zone chain.
- **The decision table missed M2, M3, M11, M12 and M15:** its tests use a single weight (4 kg) and a single parcels value per rule, so no threshold is touched.

Each technique is blind in a different place, and the places do not overlap: the union of the black-box techniques found every seeded fault, which is why one technique alone is not enough.

Interpretation: which approach found more?

Counted by defects, the black-box tests found more: the real defect, all 18 seeded faults, against 13 of 18 and no real defect for the white-box tests. The important defect, the one that loses money for customers who mistype a zone, was found by one black-box test. But the right conclusion is not “black-box is better”. Two facts qualify it. First, the black-box tests also reached 100% of the white-box criteria: the 7 ECP tests alone cover all 19 statements, 14 branches and 16 condition outcomes, whereas the 8 basis-path tests cover only 10 of the 12 equivalence classes (the invalid zone and weight above 50 are missing; BP9 adds the second). So structure-driven tests are a subset of what specification-driven tests already exercised, not the other way round. Second, the white-box work had value that the count does not show: it gave the proof that the code has no unexecuted statement or branch, and it showed that the defect is an omission and why a coverage figure cannot find it.

Overlapping tests that can be removed

The four techniques produced 34 tests (7 + 12 + 6 + 9). Six were exact duplicates of each other (for example weight 0, zone A appears in four sets), leaving 28 distinct tests. Choosing the smallest set that keeps every one of these: all 18 seeded faults, the real defect, 100% statement/branch/condition coverage, all 12 classes, all 6 decision-table rules, all 12 boundary values and the loop cases, leaves **14 tests**. So **20 of the 34 tests (59%) can be removed with no loss** of defect-finding power; the retained 14 are listed in Task 11.

Would the result hold for other modules?

Table 25: Where each approach is likely to be stronger

Kind of module	Likely stronger	Reason
shipCharge (precise rules, 19 lines)	Black-box, as found	The main defect is an omission, and thresholds are in the specification
Parcel tracking (state-based)	Black-box (state model)	The behaviour is in the model of states and guards; code structure is rarely the cause of illegal transitions
Algorithm with many nested conditions or loops (e.g. tax or route calculation)	White-box adds more	Structure creates many paths and conditions that no specification enumerates; condition and MC/DC coverage reveal untested logic
Input validation or form handling	Black-box	Missing validation is again an omission
Legacy code with no specification	White-box first	No specification to derive classes from; coverage shows what has not been run

The result for shipCharge would therefore **not hold in general**: it is favoured by a short, exact specification, a small code body and an omission-type defect. For a more complicated algorithm I expect white-box testing to find defects that no black-box test would target. The sensible conclusion is the complementary use of both, at a cost that Task 11 shows to be small.

Task 11 – Recommend and Defend the Final Test Suite **4 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- The test cases you include and the technique each comes from
- Why this suite is better than the alternatives
- The coverage and the rule combinations it achieves
- The number of test cases and the effort needed to run them
- Known gaps and risks that remain

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

Ans:

The Final Regression Suite for SwiftShip

The final suite has two parts: **14 shipping-charge tests** chosen from the black-box and white-box designs by removing every test that did not add a defect, class, rule, boundary or coverage

item (Task 10), and **18 tracking tests** from Task 6. In all, 32 tests.

Table 26: Shipping-charge tests in the final suite (FT1–FT14), the technique each comes from and its result

ID	Weight	Zone	Express	Parcels	Technique	Expected result (from S1–S4)	Actual	Result
FT1	0.0	A	No	0	ECP / BVA / DT / BP	INVALID (weight out of range)	INVALID	Pass
FT2	50.1	A	No	0	BVA / basis path	INVALID (weight out of range)	INVALID	Pass
FT3	5.0	D	No	0	ECP	INVALID (zone not A, B or C)	₹200	FAIL
FT4	0.1	A	No	0	BVA	1 kg × 40 = ₹40	₹40	Pass
FT5	1.1	A	No	0	BVA	2 kg × 40 = ₹80	₹80	Pass
FT6	30.0	A	Yes	0	BVA	30 kg × 40 = ₹1,200; × 1.5 = ₹1,800	₹1,800	Pass
FT7	30.1	A	Yes	0	BVA	NOT_ALLOWED (express above 30 kg)	NOT_ALLOWED	Pass
FT8	30.1	A	No	0	BVA	31 kg × 40 = ₹1,240	₹1,240	Pass
FT9	49.9	A	No	0	BVA	50 kg × 40 = ₹2,000	₹2,000	Pass
FT10	50.0	A	No	0	BVA	50 kg × 40 = ₹2,000	₹2,000	Pass
FT11	10.0	A	No	9	BVA	10 kg × 40 = ₹400	₹400	Pass
FT12	4.0	B	Yes	10	Decision table	4 kg × 60 = ₹240; × 1.5 = ₹360; × 0.9 = ₹324	₹324	Pass
FT13	1.0	C	No	0	Basis path	1 kg × 90 = ₹90	₹90	Pass
FT14	1.0	A	No	10	Basis path	1 kg × 40 = ₹40; × 0.9 = ₹36	₹36	Pass

Of these, FT3 fails against the pseudocode (the real defect); the other 13 pass. Techniques: ECP 2 (FT1, FT3, with FT1 shared by BVA, the decision table and basis paths), BVA 9 (FT2, FT4–FT11), decision table 1 (FT12) and basis paths 2 (FT13, FT14; FT2 doubles as BP9). The tracking tests are VT1–VT4, RT1–RT3, CT2–CT4 and IV1–IV8 (Task 6).

Why it is better than the alternatives

Table 27: The alternatives compared (seeded defects out of 18)

Candidate suite	Tests	Seeded defects found	Real defect found?	Comment
Random, 1,000 tests (zones A–C)	1,000	16.4 (mean)	No	Needs an oracle; misses edges such as exactly 50.0 and 30.0
ECP only	7	11	Yes	Misses all threshold faults
BVA only (from the rules)	12	14	No	Never varies the zone
Worst-case BVA (range extremes)	25	6	No	Large and weak
Decision table only	6	13	No	Single weight value

Candidate suite	Tests	Seeded defects found	Real defect found?	Comment
Basis paths + BP9	9	13	No	Cannot see omissions or edges
All techniques, duplicates removed	28	18	Yes	Redundant: half the tests add nothing
Final suite	**14**	**18**	**Yes**	**Everything the 28 find, with half the tests**

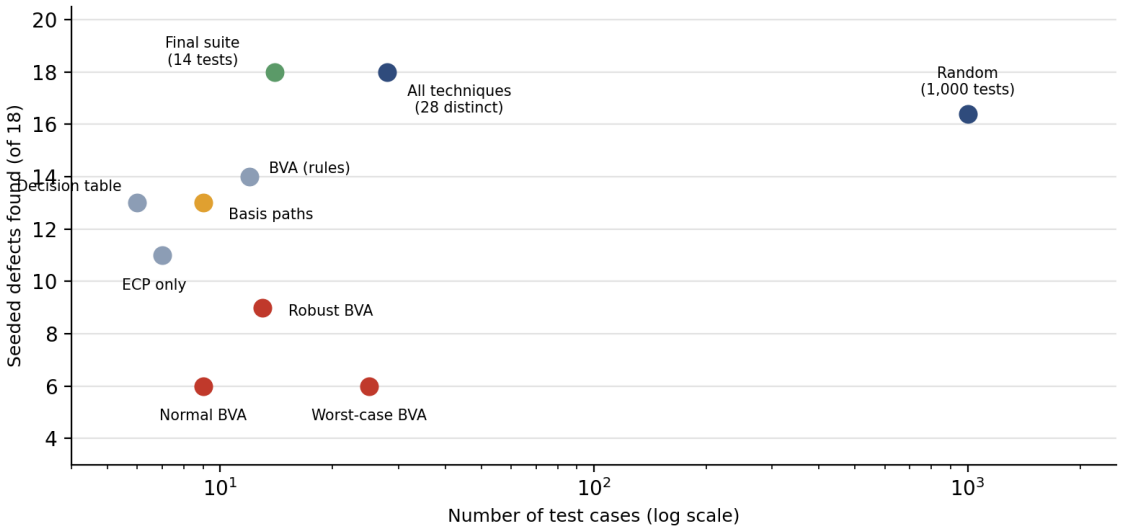


Figure 6: Tests needed against defects found, for every strategy tried

The comparison shows that the number of tests is not the measure: 25 worst-case tests find 6 seeded defects and 14 chosen tests find 18. The final suite sits on the top-left of Figure 6: the fewest tests for the most defects.

Coverage and rule combinations achieved

Table 28: What the final shipCharge suite covers

Measure	Achieved	Note
Statement coverage	19 / 19 (100%)	
Branch coverage	14 / 14 (100%)	
Condition coverage	16 / 16 (100%)	Includes all 3 combinations of line 1
Equivalence classes (Task 3)	12 / 12	Including the invalid zone
Boundary values (Task 4)	12 / 12	All weight and parcels edges
Decision-table rules (Task 5)	6 / 6	R1: FT1; R2: FT7; R3: FT6; R4: FT12; R5: FT4, FT8; R6: FT14
Basis paths (Task 8)	8 / 8	Loop passes in the suite: 1, 2, 4, 10, 30, 31 and 50

Measure	Achieved	Note
Seeded defects found	18 / 18	The only equivalent mutant (M19) cannot be found by any test
Real defect	Found	FT3
Tracking transitions	8 / 8 valid, 12 of 56 invalid	Selected by risk (Task 6)

Number of tests and effort to run them

The 14 charge tests are pure function calls and need no set-up: through an API they take about 0.5 s each (7 s in all). The 18 tracking tests must create a parcel and drive it through several states, which I estimate at 8 s each (144 s). **The whole suite of 32 tests runs in about 2.5 minutes**, and even if every charge test were run through the booking page at 20 s and tracking at 45 s per test it would take 18 minutes. Writing the expected results is a one-time effort; maintenance is the update of the expected results when S1–S4 change (Task 12).

Known gaps and risks that remain

- **Specification gaps:** more than one decimal in the weight, case of the zone code, negative parcels and rounding of the amount are not specified, so no expected result exists for them.
- **Zone × rule:** each decision-table rule is run with one or two zones only (reasonable today, not once rules depend on the zone).
- **Invalid transitions:** 44 of the 56 invalid transitions are not tested; I assume that they share the same transition check as the 12 that are.
- **Seeded, not real defects:** the 18 mutants are my choice; a minimal suite tuned to them may miss other faults, so I added two tests (FT7, FT11) for edges that more than one mutant targets.
- **Not covered at all:** concurrency (two bookings for the same parcel), performance with 40,000 bookings a day, the web page and the tracking code, which I have not seen (only the state model was tested).
- **Oracle:** the expected values are my reading of S1–S4; where the analyst disagrees they must be re-derived.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Test Suite **8 Marks**

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- What has changed in the specification and the code
- Which equivalence classes, boundaries and decision-table rules are affected
- How the control flow graph and cyclomatic complexity change
- Why the 30-minute limit matters

Evidence Expected

A clear analysis of how the changes affect the original test design.

Ans:

Impact of the Fuel Surcharge and Weekend Delivery

What has changed

Table 29: Specification and code before and after

Item	Before	After (my reading)
Specification	S1–S4	S5: Zone C parcels pay an 8% fuel surcharge. S6: a weekend-delivery option adds ₹50 per parcel
Inputs	weight, zone, express, parcels	A fifth input, weekend (yes/no): every call changes its signature
Code	19 lines	Two new decisions after line 16: if zone == "C" then charge = charge * 1.08; if weekend then charge = charge + 50
Order of calculation	base, Express, discount	base, Express, **surcharge, weekend fee**, discount. So the surcharge applies to the Express-inclusive charge and the discount reduces the weekend fee as well
Result strings	INVALID, NOT_ALLOWED, amount	Unchanged

The order is my interpretation of "after line 16" and must be confirmed with the analyst: if the ₹50 were meant to be outside the discount, the code would be wrong for 10%-discount customers by ₹5. I also assume the two decisions were put at the main level, between line 16 and line 17, so they apply to every parcel. If they were typed one indentation deeper, inside the Express block, the surcharge and the fee would apply to Express parcels only; my tests are designed to expose that placement mistake (NT1 in Task 12C).

What is affected in the test design

Table 30: Test-design items affected by the change

Item	Effect	Detail
Equivalence classes	One new variable, one class changes	Weekend: Yes / No (two new valid classes). Zone C (EZ3) now has a different result, so every expected value for Zone C changes
Boundary values	Unchanged	No new numeric threshold; weight and parcels boundaries are as before. The “edge” that is new is the order of ₹50 and the discount, tested by a rule combination
Decision table	Grows from 6 to 18 rules	The charge branch is decided by four independent yes/no factors: Express, Zone C, Weekend, Discount = 16 rules, plus INVALID and NOT_ALLOWED
State-transition tests	Unchanged	No state is added. I note that a weekend delivery could influence delivery attempts; if the tracking rules change, this model must be revised
Expected results	Change for Zone C	Every valid Zone C test is ×1.08, and every test needs weekend = No added

Control flow graph and cyclomatic complexity

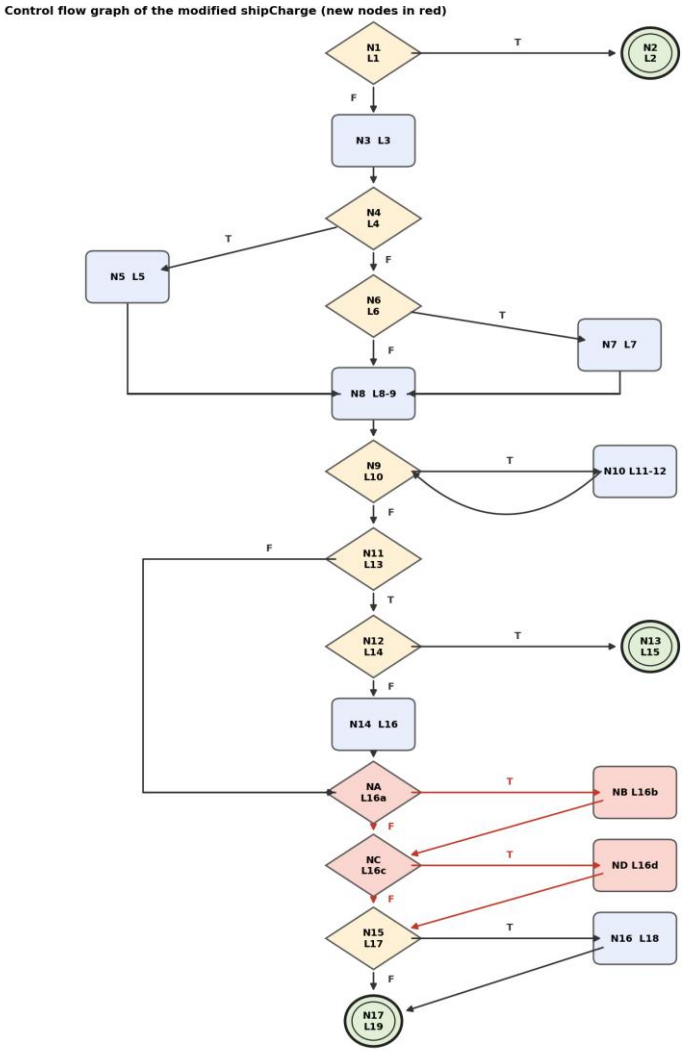


Figure 7: Control flow graph of the modified shipCharge

The graph gains four nodes: NA (the decision on zone C, line 16a), NB (the surcharge), NC (the weekend decision) and ND (the fee). The two express branches now join before NA instead of before N15, so the edges N14 → N15 and N11 → N15 are replaced by N14 → NA and N11 → NA. The edge count rises from 24 to 30 and the node count from 18 to 22.

Table 31: Complexity before and after

Measure	Before	After	Check
Edges E	24	30	counted from the graph
Nodes N (with EXIT)	18	22	4 new nodes
V(G) = E − N + 2	8	10	30 − 22 + 2
V(G) = decisions + 1	8	10	7 + 1 and 9 + 1
With line 1 split (N1a, N1b)	9	11	one more node, two more edges

Measure	Before	After	Check
Basis paths needed	8	10	rank of the path vectors verified for all feasible paths
Feasible paths in all	541	1,021	3 zone routes × 340 + INVALID

The two basis paths added are: **weekend = Yes** (the new fee statement), and **Zone C with Express above 30 kg**, which is NOT_ALLOWED. The second one is not obvious: before the change the zone C and the NOT_ALLOWED edges were never needed together, but the new decision NA is only reached by parcels that pass the Express check, so a zone C parcel that leaves at N13 is now a path of its own. I checked by computing the rank of the edge vectors of all feasible paths: it is 10. It also shows that the first new decision (zone C) repeats the condition of line 6, so the two cannot be changed independently by a test.

Why the 30-minute limit matters

A suite that does not finish in 30 minutes will not be run before every release, or will be cut by hand under deadline pressure, and then defects like those of Task 10 reach customers. The time is the number of tests multiplied by the time of one test, and the number of tests grows with every new input: one more yes/no input doubles the rule combinations (6 → 16 with two new factors). Worst-case BVA, with the new input, would need 25 × 3 zones × 2 × 2 = 300 charge tests: 5 minutes through an API, but **114 minutes** if tests go through the booking page (Figure 8). The limit therefore forces a choice of tests by value, not by volume.

B. Evaluation of the Original Suite – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- Which existing tests are still valid, need new expected results, or are obsolete
- Which new paths and rule combinations are not covered
- Whether the coverage criterion can still be met

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

Ans:

Is the Original Suite Still Adequate?

Table 32: Status of each original test under the changed specification

Test	Inputs	Status
FT1	0.0 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT2	50.1 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No

Test	Inputs	Status
FT3	5.0 kg, zone D, Express No, 0 parcels	Valid; add Weekend = No
FT4	0.1 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT5	1.1 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT6	30.0 kg, zone A, Express Yes, 0 parcels	Valid; add Weekend = No
FT7	30.1 kg, zone A, Express Yes, 0 parcels	Valid; add Weekend = No
FT8	30.1 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT9	49.9 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT10	50.0 kg, zone A, Express No, 0 parcels	Valid; add Weekend = No
FT11	10.0 kg, zone A, Express No, 9 parcels	Valid; add Weekend = No
FT12	4.0 kg, zone B, Express Yes, 10 parcels	Valid; add Weekend = No
FT13	1.0 kg, zone C, Express No, 0 parcels	New expected result: ₹90 → ₹97.20 (Zone C × 1.08)
FT14	1.0 kg, zone A, Express No, 10 parcels	Valid; add Weekend = No

Summary: 13 tests remain valid, 1 needs a new expected result (FT13, zone C: ₹90 becomes ₹97.20), 0 are obsolete, and all 14 need the new input (weekend = No). FT3 still reveals the invalid-zone defect. No test is obsolete because no rule was removed; the tests that return INVALID or NOT_ALLOWED are unaffected, as those results are produced before the new code is reached.

Paths and combinations not covered by the retained tests

I ran the 14 tests (with weekend = No) against the new code and measured: **statement coverage 22 of 23, branch coverage 17 of 18** (the weekend fee statement and the weekend-true branch are never executed), and the tests found only **23 of 27** seeded faults. The faults missed, N3, N5, N8, N9, are: a wrong fee (₹5 instead of ₹50), a fee applied only when parcels are below 10, the fee added after the discount instead of before it, and the surcharge computed on the base charge instead of on the Express-inclusive charge. The retained tests also cover only 16 of the 24 possible pairs of values for the four factors (Express, Zone C, Weekend, Discount) and only 5 of the 16 rule combinations, and not one has Zone C together with Express.

Can the coverage criterion still be met?

Yes, at low cost. The criterion of Task 9 (100% branch and condition coverage) needs only **one** more test, one with weekend = Yes: it reaches 23 of 23 statements and 18 of 18 branches. The specification criteria (classes, rules) cannot be met without the new factors: Weekend = Yes is a new class, and the rule combinations need at least the new tests of Task 12C.

A new finding: the amount is not exact

With the new factors some results are no longer exact decimals. For Zone C, Express, 10 parcels, weight 4.0 kg, the correct charge is ₹524.88 but the program returns 524.8800000000001 (floating-point multiplication by 1.08); with a weekend fee the same happens (₹569.88). The specification has no rounding rule. Printed to two decimals the value

is correct, but a test comparing for equality fails, and a total over thousands of parcels could differ from the ledger by a few paise. I therefore recommend a rounding rule in the specification (to the nearest paisa) and tests that compare amounts to the paisa, not as floating-point values.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

Discuss:

- Describe the new and changed test cases
- Explain which techniques you used to derive them
- Show the effect on coverage and the number of test cases

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

Ans:

Decision: MODIFY the Test Suite

Table 33: Retain, modify or redesign

Option	What it would mean	Verdict
Retain	Run the old 14 charge tests unchanged (only add weekend = No)	Rejected: finds 23 of 27 faults, never executes the fee statement, and one expected result is already wrong
Modify	Keep the 14 tests, correct FT13, add the few tests that the new factors need	Chosen: all 27 faults found; 20 charge tests
Redesign	Throw the suite away and derive a new one from the new specification	Rejected: the specification changed by two factors only; 13 tests are still valid and the techniques did not become wrong

Changes. All 14 charge tests receive weekend = No; FT13 gets its new expected result (₹97.20); and **six new tests** are added. The tracking tests (18) are unchanged. The new tests were derived as follows: decision-table thinking gave the four factors (Express, Zone C, Weekend, Discount); choosing tests so that every **pair** of values of two factors occurs at least once (24 pairs) gave the combinations; basis-path analysis demanded a weekend = Yes path (NT1) and a Zone C NOT_ALLOWED path (NT6); and the seeded-fault check showed which combinations are needed to expose the order and placement faults.

Table 34: New tests NT1–NT6

ID	Weight	Zone	Express	Parcels	Weekend	Purpose	Expected result (changed rules)	Actual	Result
NT1	4.0	A	No	5	Yes	Weekend fee only (zone A): the new fee statement	4 kg × 40 = ₹160; + 50 = ₹210	₹210	Pass
NT2	4.0	B	No	10	Yes	Weekend with discount and zone B: is the ₹50 discounted?	4 kg × 60 = ₹240; + 50 = ₹290; × 0.9 = ₹261	₹261	Pass
NT3	4.0	C	No	10	Yes	Zone C with weekend and discount, no Express	4 kg × 90 = ₹360; × 1.08 = ₹388.80; + 50 = ₹438.80; × 0.9 = ₹394.92	₹394.92	Pass
NT4	4.0	C	Yes	5	No	Zone C with Express: surcharge on the Express-inclusive charge	4 kg × 90 = ₹360; × 1.5 = ₹540; × 1.08 = ₹583.20	₹583.20	Pass
NT5	4.0	C	Yes	10	Yes	All four factors together: Express, Zone C, Weekend, Discount	4 kg × 90 = ₹360; × 1.5 = ₹540; × 1.08 = ₹583.20; + 50 = ₹633.20; × 0.9 = ₹569.88	₹569.88	Pass
NT6	35.0	C	Yes	12	Yes	Express above 30 kg, zone C, weekend: NOT_ALLOWED comes first	NOT_ALLOWED (express above 30 kg); no surcharge or weekend fee	NOT_ALLOWED	Pass

All six pass the modified code, apart from the rounding remark of Task 12B. In the seeded-fault check NT1 finds the fee typed inside the Express block and a wrong fee amount; NT2 finds the surcharge applied to zone B as well and the fee added after the discount; NT3 finds the surcharge placed on zone B instead of zone C and a fee limited to parcels below 10; NT4 finds the surcharge computed on the base charge instead of on the Express-inclusive charge; and NT6 finds no seeded fault but gives the new path (zone C, NOT_ALLOWED) that the basis-path analysis requires.

Effect on coverage and the number of tests

Table 35: Original suite adapted by retention, compared with the modified suite

Measure	Retained (14 tests)	Modified (20 tests)	Full 16 rules (30 tests)
Charge tests	14	20	30
Tracking tests	18	18	18
Statement coverage (23 statements)	22 / 23	23 / 23	23 / 23
Branch coverage (18 branches)	17 / 18	18 / 18	18 / 18
Seeded faults found (27)	23 / 27	27 / 27	27 / 27

Measure	Retained (14 tests)	Modified (20 tests)	Full 16 rules (30 tests)
Factor pairs covered (24)	16 / 24	24 / 24	24 / 24
Rule combinations covered (16)	5 / 16	10 / 16	16 / 16
Basis paths covered (10)	8 / 10	10 / 10	10 / 10
Runtime, API (18 tracking tests included)	2.5 min	2.6 min	2.6 min
Runtime, all through the web page	18.2 min	20.2 min	23.5 min

The modified suite reaches full statement and branch coverage, finds every non-equivalent seeded fault and covers every pair of factor values with 6 more tests. It leaves 6 of the 16 rule combinations untried; I accept this because the factors act as independent multiplications and additions, so an interaction fault would have to be one of the ordering faults tested by NT4–NT5. If the team prefers the full 16 rules (30 tests), it still takes 2.6 minutes through an API and is a reasonable choice once the rounding rule is settled.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Discuss:

- Equivalence classes and boundaries
- Decision-table combinations
- Cyclomatic complexity and basis paths
- Coverage achieved
- Execution time limit

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

Ans:

Defending the Modified Suite

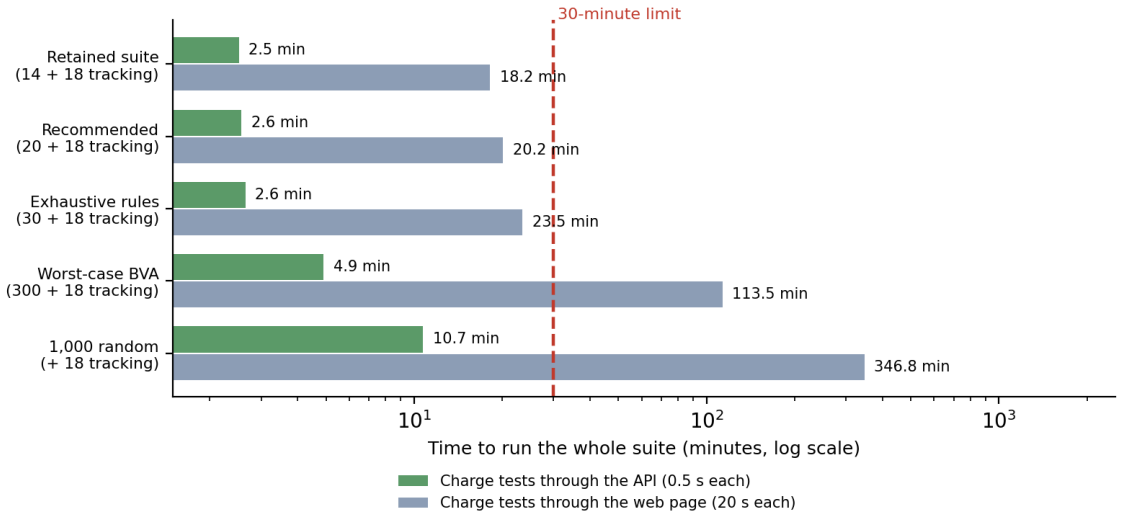


Figure 8: Time needed to run each candidate suite against the 30-minute limit

Table 36: Evidence for the decision to modify the suite

Evidence	What it shows	Effect on the decision
Equivalence classes and boundaries	Weekend adds two classes; no new numeric edge. All 12 original classes and 12 boundaries are still valid, and 13 of 14 tests keep their expected results	Keep the old tests; add the weekend class
Decision-table combinations	Four independent factors give 16 charge rules. The retained tests cover 5; the modified suite 10 and every pair of values (24 of 24)	Add combinations chosen by pairs and by ordering risk
Cyclomatic complexity and basis paths	V(G) rises from 8 to 10; two paths are new (weekend, zone C NOT_ALLOWED); 1,021 paths exist, so only the independent ones can be tested	Add NT1 and NT6
Coverage achieved	Retained: 22/23 statements, 17/18 branches, 23/27 seeded faults. Modified: 23/23, 18/18, 27/27	The adequacy gate of Task 9 is met again
Execution time	The modified suite of 20 + 18 tests takes 2.6 minutes (20 minutes if every test went through the web page), well within 30 minutes. Worst-case BVA (300 charge tests) would take 114 minutes through the web page	The suite fits the limit (about 90% of the time to spare through an API, a third through the web page), so there is no need to cut tests
Real defect	FT3 (invalid zone) still fails in the old and the new code	The defect must be fixed before the new feature is released

Why not retain. It leaves the new code untested at the places where the change was made: the ₹50 fee is never executed, and a developer who puts the fee after the discount, or inside the Express block, would not be caught. The tests that remain are also not entirely correct, as FT13 gives a wrong expected result for Zone C.

Why not redesign. Nothing in the original design was wrong: the classes, boundaries and rules are unchanged except for the new factors, and 13 of 14 tests are still correct. A redesign would cost more time than the whole suite takes to run, and would discard the evidence that each of the 14 tests earns its place.

Why 20 tests and not 30 or 300. The modified suite finds every seeded fault and covers every pair of factor values. The extra 10 tests of the full table add certainty for combinations that the code treats independently, and the 300 of worst-case BVA add nothing that the 20 miss. Both would fit the time limit through an API; the choice between them is a question of maintenance effort, not of the limit.

Residual risks and the trigger to revisit. (1) The order of the surcharge, the fee and the discount follows my reading of "after line 16": if the analyst intends otherwise, NT2, NT3 and NT5 change at once. (2) No rounding rule exists. (3) The tracking model assumes that weekend delivery adds no state. I would revisit this decision, and move to a larger suite, if a later defect report shows a fault in a combination that the pairs did not cover, or if the suite approaches 30 minutes when run through the user interface.

MARK DISTRIBUTION

Task	Assessed area	Marks
Task 1	Select Test Case Design Strategies	5
Task 2	Evaluate Random Testing	2
Task 3	Design and Evaluate Equivalence Classes	6
Task 4	Design and Evaluate Boundary Value Tests	4
Task 5	Apply Other Black-Box Techniques	3
Task 6	Design and Evaluate State-Transition Tests	3
Task 7	Construct the Control Flow Graph	4
Task 8	Evaluate Path Testing and Cyclomatic Complexity	4
Task 9	Judge Test Adequacy and Coverage	3
Task 10	Compare Black-Box and White-Box Effectiveness	4
Task 11	Recommend and Defend the Final Test Suite	4
Task 12	Day-4 Design Challenge (A–D, 2 marks each)	8
Total		50